

Parecer de Auditoria Técnico-Científica

Conformidade Arquitetural, Segurança (ISO/IEC 27001) e Privacidade (LGPD)

Auditoria de Segurança e Arquitetura de Software

3 de junho de 2026

Resumo

Este documento apresenta o parecer técnico-científico e a auditoria de segurança da informação referente ao projeto **CommunitySafe.Api**, desenvolvido sob a stack tecnológica C# 12 e .NET 8. A análise avalia a aderência do sistema aos pilares de Confidencialidade, Integridade e Disponibilidade (CID), em conformidade com as diretrizes da norma ISO/IEC 27001:2022 e da Lei Geral de Proteção de Dados (LGPD - Lei nº 13.709/2018). Foram inspecionados mecanismos de autenticação (JWT, TOTP), criptografia (AES-256, BCrypt), design arquitetural orientado a domínio e resiliência contra vetores de ataque modernos.

Conteúdo

1	Autenticação, MFA e Criptografia Aplicada	2
1.1	Gestão de Senhas e Hashing	2
1.2	Múltiplos Fatores de Autenticação (MFA)	2
1.3	Criptografia de Dados Estacionários (Data at Rest)	2
2	Arquitetura, Injeção de Dependências e Padrões de Projeto	2
3	Recuperação de Senha, OTP e Notificações (Sns, Email)	2
4	Gestão de Ativos, LGPD e Privacidade (Privacy by Design)	3
5	Matriz de Conformidade Técnica	3
6	Threat Modeling (Modelagem de Ameaças) e Remediações	4
6.1	Mapeamento de Riscos (Vulnerabilidades e Ameaças)	4
6.2	Remediações e Código Corretivo C# (Secure Coding)	4
7	Conclusão do Parecer de Auditoria	5

1 Autenticação, MFA e Criptografia Aplicada

A análise da camada de autenticação do *CommunitySafe.Api* revela uma implementação robusta utilizando o protocolo JWT (JSON Web Token) associado a algoritmos de *hashing* assimétrico e criptografia simétrica.

1.1 Gestão de Senhas e Hashing

A classe `PasswordHasher` faz uso da biblioteca `BCrypt.Net-Next`. O uso do `BCrypt` é cientificamente endossado por incorporar um fator de trabalho (*work factor*) que mitiga ataques de força bruta iterativos e mitigação de *rainbow tables* devido à geração nativa de *salts* únicos por iteração. A escolha tecnológica atende ao Requisito 1.1 e 1.2 da matriz de conformidade.

1.2 Múltiplos Fatores de Autenticação (MFA)

O sistema suporta autenticação em duas etapas (2FA) de maneira modular:

- **TOTP (Time-Based One-Time Password):** Implementado via `TotpService` utilizando `Otp.NET`. É aderente à RFC 6238, gerando tokens baseados em janela de tempo (30 segundos), reforçando a proteção contra ataques de interceptação (*replay attacks*).
- **OTP via Mensageria (SMS/Email):** Gerenciado pelo `EmailOtpService` e `AwsSnsSmsService`, com controle rigoroso de expiração e invalidação após o uso.

1.3 Criptografia de Dados Estacionários (Data at Rest)

A classe `AesEncryptionService` demonstra a aplicação de *Advanced Encryption Standard* (AES) operando em modo CBC (Cipher Block Chaining). O IV (Initialization Vector) é corretamente gerado de forma aleatória para cada operação de encriptação, garantindo a difusão criptográfica.

2 Arquitetura, Injeção de Dependências e Padrões de Projeto

A aplicação adota uma variação do *Clean Architecture* e *Onion Architecture*, separando claramente as responsabilidades.

- **Inversão de Controle (IoC):** O contêiner de injeção de dependência nativo do .NET 8 é utilizado extensivamente (`Program.cs`). Serviços como `IAuthService`, `IEventService` e `IEmailService` são mapeados como dependências *Scoped* ou *Transient*, o que otimiza o ciclo de vida da memória e previne vazamentos (*memory leaks*).
- **Data Transfer Objects (DTOs):** A utilização de `AuthDtos.cs` e `RegisterDtos.cs` atua como uma barreira de proteção anti-corrupção, mitigando ataques de *Mass Assignment* (Atribuição em Massa) ou *Over-posting*, impedindo que estruturas internas do banco de dados sejam manipuladas diretamente por entradas HTTP.

3 Recuperação de Senha, OTP e Notificações (Sns, Email)

O processo de recuperação de credenciais e comunicação é coordenado através do `OtpDispatcher`, que delega a execução para os respectivos canais (E-mail via `MailKit/SMTP` ou SMS via `AWS SNS`).

1. **Geração Segura:** O OTP gerado é estocástico e possui um limite de tempo estrito definido nas configurações de ambiente (`appsettings.json`), mitigando a janela de vulnerabilidade.
2. **Trilha de Auditoria:** Serviços de disparo (como `AwsSnsSmsService`) possuem telemetria encapsulada (embora logs de erro exijam supressão de PII para evitar vazamento de informações sensíveis nos logs).
3. **Invalidação Garantida:** Seguindo os requisitos funcionais, os tokens de recuperação são projetados para uso único (Nonce).

4 Gestão de Ativos, LGPD e Privacidade (Privacy by Design)

Sob a ótica da Lei Geral de Proteção de Dados (LGPD) e das atribuições de um *Data Protection Officer* (DPO), a solução demonstra aderência ao princípio de *Privacy by Design*.

- **Minimização de Dados:** Os DTOs de registro recolhem dados estritamente necessários à finalidade comunitária. Validações estritas de domínio (CEP via `CepLookupService` e documentos via `DocumentValidator`) garantem a integridade dos dados na entrada.
- **Pseudonimização e Anonimização:** Através do `AesEncryptionService`, atributos identificáveis (PII) podem ser armazenados de forma segura, garantindo que o banco de dados principal no PostgreSQL sofra exfiltração sem comprometer a legibilidade imediata das PIIs (Confidencialidade).
- **Direitos do Titular:** O *Controller* `MeController` fornece *endpoints* para recuperação e governança dos dados associados ao JWT da requisição autêntica, atendendo aos direitos de acesso aos dados (Art. 18, LGPD).

5 Matriz de Conformidade Técnica

A tabela a seguir consolida o *compliance* cruzado entre os requisitos levantados (conforme PDF de requisitos) e as evidências técnicas encontradas no código fonte.

Categoria de Requisito	Evidência Técnica no Projeto	Status
1. Gestão de Senhas e Hash	Utilização de <code>BCrypt.Net-Next</code> no serviço <code>PasswordHasher</code> com salt autogerenciado.	Conforme
2. Algoritmo Multi-fator (2FA)	<code>TotpService</code> configurado via <code>Otp.NET</code> e despachantes OTP para SMS e Email.	Conforme
3. Proteção contra Brute-Force	Exige acoplamento futuro de Rate Limiter nativo do .NET 8 no <code>Program.cs</code> .	Parcial
4. Validação e Criptografia	<code>AesEncryptionService</code> para <i>data at rest</i> e <code>DocumentValidator</code> para consistência.	Conforme
5. Privacidade (LGPD)	DTOs para minimização de dados e barreiras para <i>Over-posting</i> .	Conforme

Continua na próxima página...

Categoria de Requisito	Evidência Técnica no Projeto	Status
6. Recuperação de Senha	Gestão baseada em <i>tokens</i> com <i>timeout</i> explícito via OTP.	Conforme

6 Threat Modeling (Modelagem de Ameaças) e Remediações

Utilizando o framework STRIDE, identificamos riscos arquiteturais residuais e propomos remediações a nível de código baseadas em *Secure Coding*.

6.1 Mapeamento de Riscos (Vulnerabilidades e Ameaças)

Risco 1: Path Traversal (Acesso Indevido a Arquivos)

Pilar Afetado: Confidencialidade e Integridade

Severidade: Alta

Impacto: Caso rotinas futuras incorporem upload/download de avatares ou relatórios, a ausência de sanitização rigorosa nos caminhos (Paths) permite leitura arbitrária de arquivos de sistema (ex: `appsettings.json`).

Risco 2: Exaustão de Recursos (DDoS em L7)

Pilar Afetado: Disponibilidade

Severidade: Média

Impacto: Inexistência de *Rate Limiting* global explícito ou limites de tamanho de corpo (*Body Size Limits*) no código analisado, permitindo que atacantes floodem as APIs com *payloads* gigantes ou *requests* de autenticação sucessivos.

6.2 Remediações e Código Corretivo C# (Secure Coding)

As implementações corretivas a seguir, em **C# .NET 8**, são de caráter mandatório para o ambiente de produção.

Listing 1: Mitigação de Path Traversal e Guardrails de Arquivos

```
1 public class SecureFileHandler
2 {
3     // 1. Mitigacao de Path Traversal
4     public string GetSecureFilePath(string baseDirectory, string untrustedFileName)
5     {
6         // Rejeita silenciosamente nomes de arquivos com caracteres invalidos
7         if (untrustedFileName.IndexOfAny(Path.GetInvalidFileNameChars()) >= 0)
8             throw new ArgumentException("Formato de arquivo inseguro.");
9
10        // Resolve o caminho canonico completo
11        string fullPath = Path.GetFullPath(Path.Combine(baseDirectory,
12            untrustedFileName));
13
14        // Valida se o caminho final ainda reside dentro do diretorio base permitido
15        if (!fullPath.StartsWith(Path.GetFullPath(baseDirectory),
16            StringComparison.OrdinalIgnoreCase))
17            throw new UnauthorizedAccessException("Alerta de Seguranca: Tentativa de
18            Path Traversal interceptada.");
19
20        return fullPath;
21    }
22 }
```

```
19 }
```

Listing 2: Implementação de Limitação de Recursos e Rate Limiting (.NET 8)

```
1 // No arquivo Program.cs (Pipeline Configuration)
2 using Microsoft.AspNetCore.Http.Features;
3 using Microsoft.AspNetCore.RateLimiting;
4 using System.Threading.RateLimiting;
5
6 var builder = WebApplication.CreateBuilder(args);
7
8 // 2. Limite rígido de Memória/Corpo da Requisição (Max 2MB)
9 builder.Services.Configure<FormOptions>(options =>
10 {
11     options.MultipartBodyLengthLimit = 2097152;
12 });
13 builder.WebHost.ConfigureKestrel(options =>
14 {
15     options.Limits.MaxRequestBodySize = 2097152;
16 });
17
18 // 3. Rate Limiter Global contra Força Bruta (DDoS L7)
19 builder.Services.AddRateLimiter(options => {
20     options.GlobalLimiter = PartitionedRateLimiter.Create<HttpContext,
21         string>(httpContext =>
22             RateLimitPartition.GetFixedWindowLimiter(
23                 partitionKey: httpContext.Connection.RemoteIpAddress?.ToString() ??
24                     "unknown",
25                 factory: partition => new FixedWindowRateLimiterOptions
26                 {
27                     AutoReplenishment = true,
28                     PermitLimit = 50, // 50 requests
29                     Window = TimeSpan.FromMinutes(1) // Por minuto
30                 }
31             ));
32 });
```

Listing 3: Trava de Integridade da Stack (Lock Files)

```
1 <!-- No arquivo CommunitySafe.Api.csproj -->
2 <Project Sdk="Microsoft.NET.Sdk.Web">
3     <PropertyGroup>
4         <!-- 4. Fixação e validação rigorosa das dependências (NPM/Nuget Supply Chain
5             Attack Mitigation) -->
6         <RestorePackagesWithLockFile>true</RestorePackagesWithLockFile>
7         <DisableImplicitNuGetFallbackFolder>true</DisableImplicitNuGetFallbackFolder>
8     </PropertyGroup>
9 </Project>
```

7 Conclusão do Parecer de Auditoria

O projeto **CommunitySafe.Api** evidencia alta maturidade arquitetural e aderência significativa a boas práticas de segurança defensiva. A utilização de *hashing* criptográfico consolidado (BCrypt), acoplado a serviços de dupla fatoração (TOTP e OTP) atende substancialmente aos controles de acesso lógicos preconizados pela ISO/IEC 27001:2022 (Controle A.9 - Controle de Acesso) e os princípios de privacidade por design exigidos pela LGPD.

Contudo, para homologação da plataforma em ambientes de produção irrestritos, é mandatório a aplicação de mitigadores de falhas na camada 7 (Rate Limiting) e travas de segurança

de Supply Chain (Lockfiles de dependências), conforme exemplificados na Seção 6. Executados esses ajustes, a arquitetura torna-se resiliente e apta para o tratamento de dados pessoais críticos.

Referências Bibliográficas

- ABNT NBR ISO/IEC 27001:2022. Segurança da informação, segurança cibernética e proteção de privacidade - Sistemas de gestão da segurança da informação - Requisitos.
- BRASIL. Lei nº 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD).
- OWASP. Top 10 API Security Risks – 2023. Disponível em: <https://owasp.org/API-Security/>
- MICROSOFT. Secure coding guidelines for ASP.NET Core. 2024.